

Sistema de Hormigas Elitista

Por: Samuel, Mario y Álvaro

A dark blue diagonal gradient bar that starts from the bottom left corner and extends towards the top right corner, covering the lower half of the slide.

Índice

1. Introducción
2. Problema del viajante
3. Comportamiento de las hormigas
4. Actualización de feromonas
5. Probabilidad de transición
6. Esquema del algoritmo
7. Estrategia elitista
8. Implementación
9. Resultados
10. Conclusiones

Introducción



Objetivo: Modelar el comportamiento de las hormigas.



Usan feromonas para conectar rutas colonias y recursos.

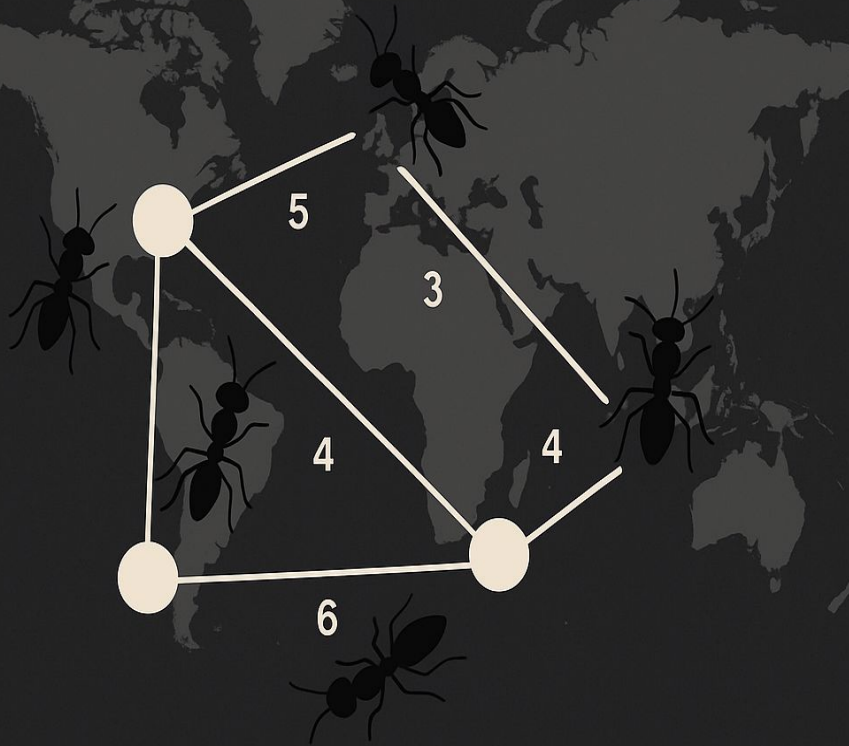
Retroalimentación positiva

Mayor
feromona



Mayor
probabilidad
de escoger
ese camino

PROBLEMA DEL VIAJANTE



Objetivo del TSP: encontrar la ruta más corta que visite todas las ciudades una sola vez y vuelva al origen

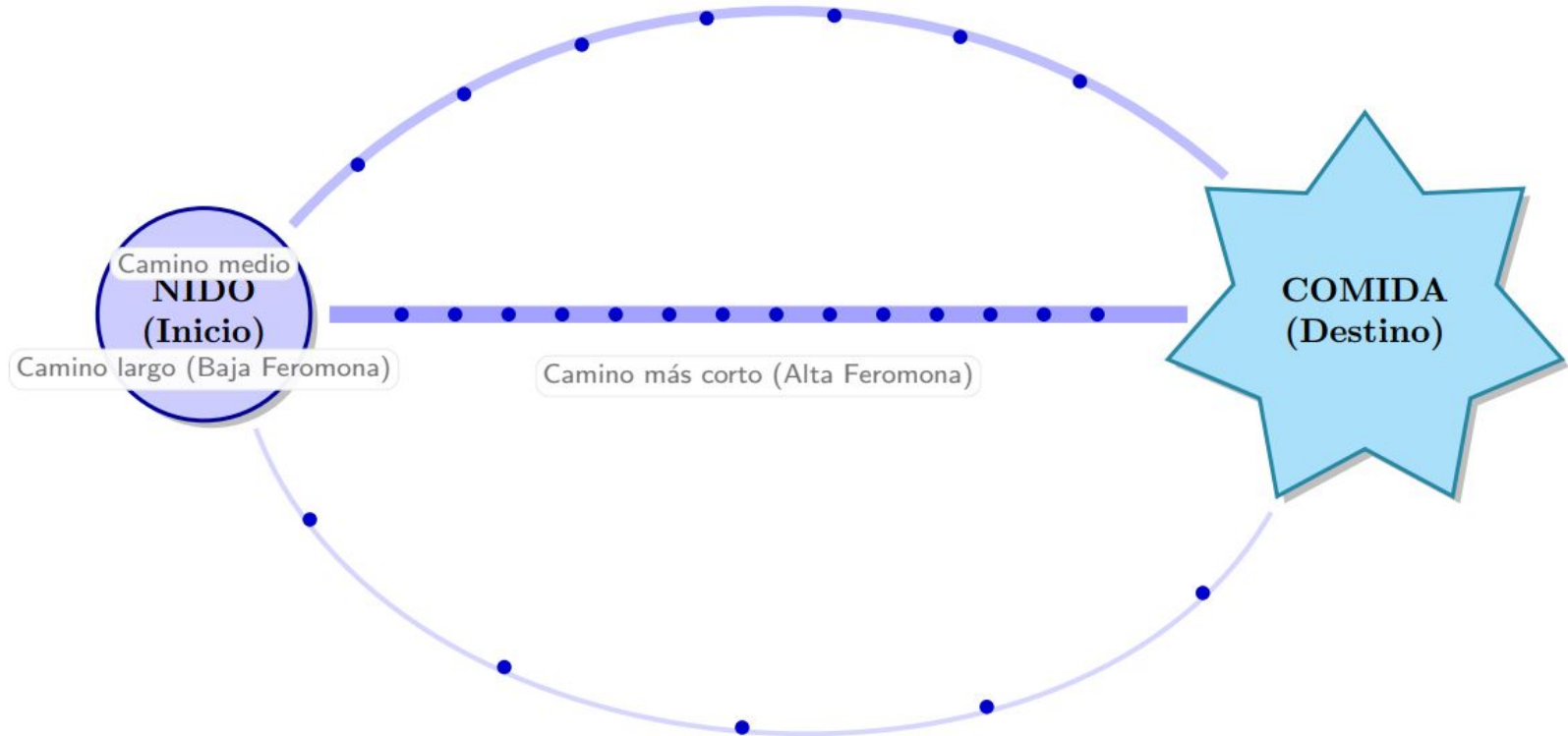
Representación como grafo:

- Nodos = ciudades
- Aristas = caminos entre ciudades con distancia d_{ij}

Se definen m hormigas repartidas por las n ciudades

Principio de Estigmergia:

Las hormigas depositan feromonas al caminar. El camino más corto se recorre más rápido, acumulando feromonas más densas, lo que atrae a más hormigas.



Comportamiento de las hormigas



Cada hormiga elige la siguiente ciudad de forma probabilística

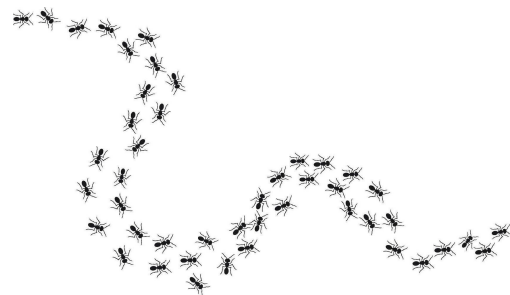


Cantidad de feromona en la arista.

Distancia (visibilidad).



Lista tabú: prohíbe volver a ciudades ya visitadas hasta completar el ciclo



Rastro de feromonas:

Al completar un recorrido, la hormiga deposita feromona en las aristas usadas

Actualización de feromonas

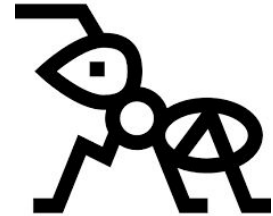


Ciclo: tras n iteraciones todas las hormigas han completado un recorrido



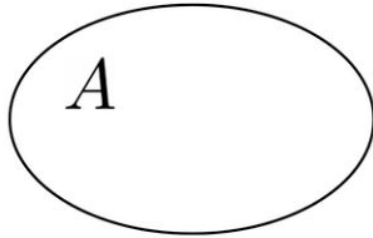
Regla de actualización global de feromona

$$\tau_{ij}(t + n) = \rho \cdot \tau_{ij}(t) + \Delta\tau_{ij}$$



$$\Delta\tau_{ij}^k = \begin{cases} Q/L_k & \text{Si utiliza } ij \\ 0 & \text{En otro caso} \end{cases}$$

Probabilidad de transición



Definimos el conjunto allowed_k :

Ciudades no visitadas por la hormiga

$$p_{ij}^k = \begin{cases} \frac{[\tau_{ij}(t)]^\alpha \cdot [n_{ij}]^\beta}{\sum_{l \in \text{allowed}_k} [\tau_{il}(t)]^\alpha \cdot [n_{il}]^\beta} & \text{si } j \in \text{allowed}_k \\ 0 & \text{en otro caso} \end{cases}$$

α : peso de la feromona

β : peso de la visibilidad
(distancia)

Esquema del algoritmo (I)

1. Inicialización
 - a. Definimos $t := 0$
 - b. Definimos $NC := 0$
 - c. Para cada arco (i, j) definimos la intensidad $\tau_{ij}(0) = c$ y $\Delta\tau_{ij} = 0$. Colocar las m hormigas en los n nodos.

2. Sea $s := 1$
 - a. Para $k = 1$ hasta m hacer
 - i. Colocar la ciudad inicial de la hormiga k en la lista tabú, $\text{tabu}_k(s)$.

3. Repetir hasta que la lista tabú se complete
 - a. Definimos $s := s + 1$
 - b. Para $k = 1$ hasta m hacer
 - i. Elegir la ciudad j a la que moverse con probabilidad $p_{kij}(t)$.
 - ii. Mover la hormiga k a la ciudad j .
 - iii. Colocar la ciudad j en la lista tabú, $\text{tabu}_k(s)$.

Esquema del algoritmo (II)

4. Para $k=1$ hasta m hacer
 - a. Mover la hormiga k de $tabu_k(n)$ a $tabu_k(1)$.
 - b. Calcular la longitud del recorrido descrito por la hormiga k , L_k .
 - c. Actualizar el camino más corto encontrado
 - d. Para cada arco (i, j) del grafo hacer

Para $k = 1$ hasta m hacer

$$\Delta\tau_{ij}^k = \begin{cases} Q/L_k & \text{si } (i, j) \in \text{ciclo descrito por } tabu_k \\ 0 & \text{en otro caso} \end{cases}$$

5. Para cada arco (i, j) calcular

$$\tau_{ij}(t + n) = \rho \cdot \tau_{ij}(t) + \Delta\tau_{ij}.$$

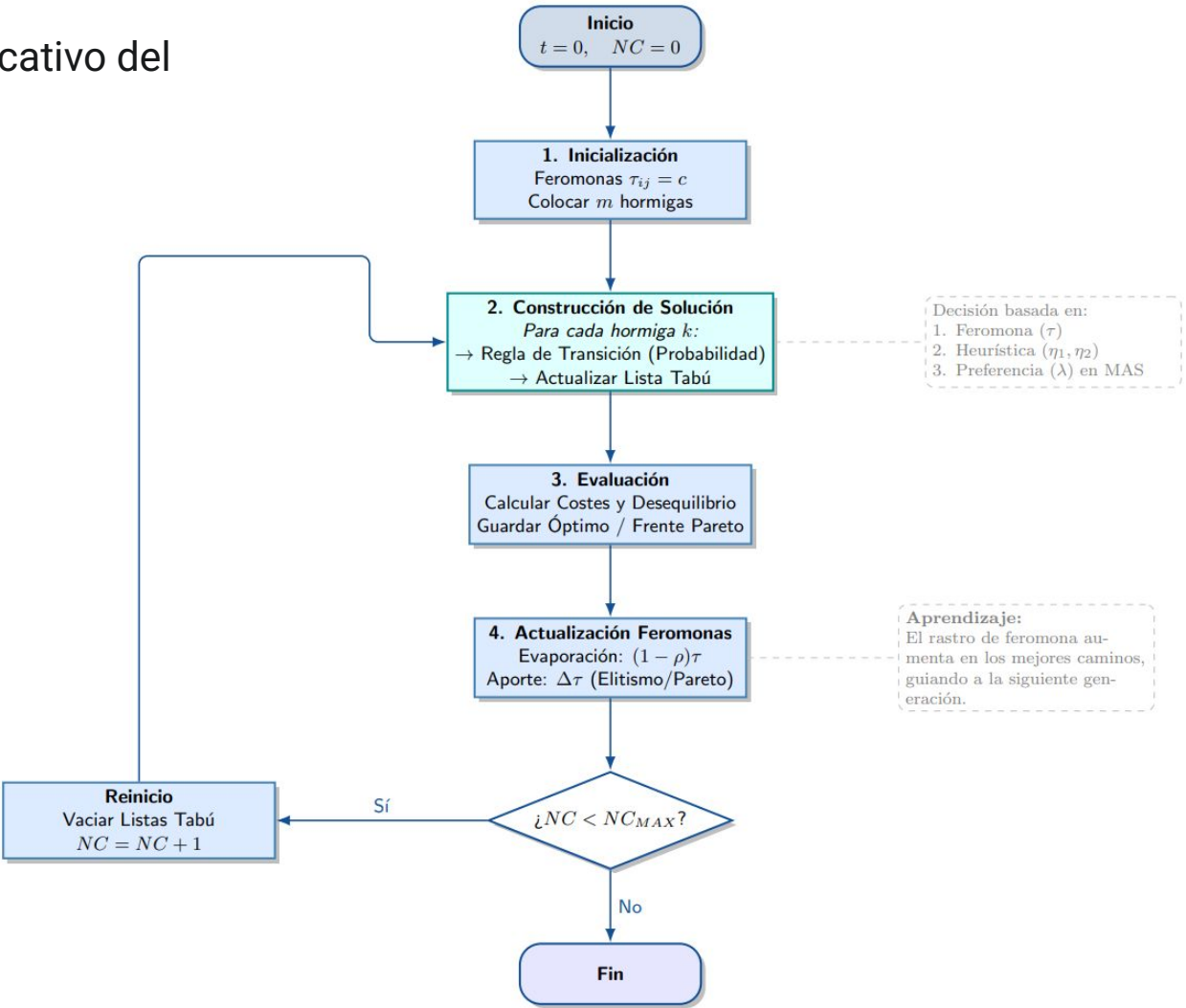
- a. Definir $t := t + n$
- b. Definir $NC := NC + 1$
- c. Para cada arco (i, j) definir $\Delta\tau_{ij} := 0$.

Esquema del algoritmo (II)

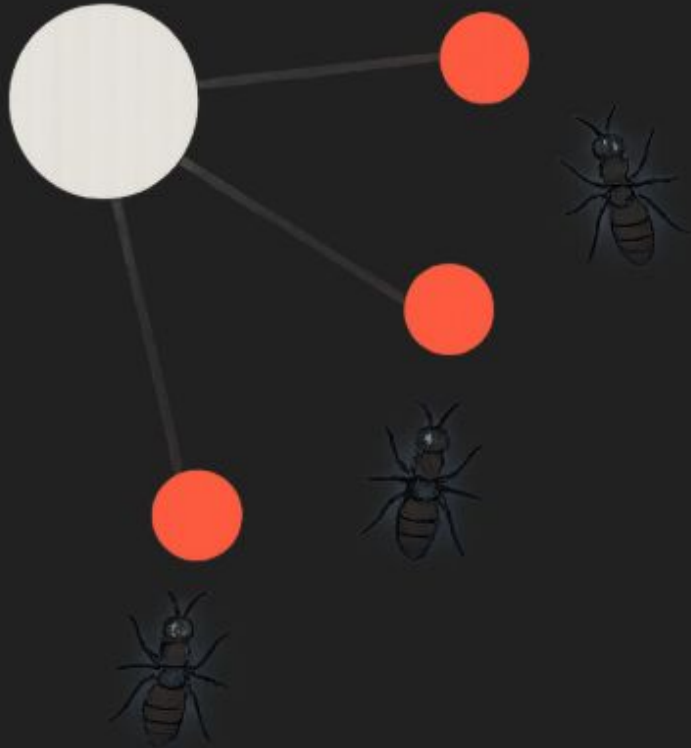
6. Si ($NC < NC_{MAX}$) y no todas las hormigas han convergido a la misma solución

- a. Entonces
 - i. Vaciar todas las listas tabú
 - ii. Ir al paso 2
- b. Si no
 - i. Imprimir el ciclo más corto encontrado
 - ii. Parar

Diagrama explicativo del algoritmo



Estrategia elitista



Mejora del rendimiento mediante refuerzo elitista

Identificamos el mejor ciclo encontrado hasta el momento (longitud L^*)

Se añade feromona extra en las aristas del mejor ciclo:

- Cantidad adicional: $e \cdot Q/L^*$
- e : número de hormigas elitistas

Efecto:

- Se refuerzan los mejores caminos
- Se acelera la convergencia del algoritmo hacia buenas soluciones

Implementación

A diferencia del TSP, aquí buscamos asignar T tareas a O operarios minimizando el coste y respetando el tiempo disponible

	T_1	T_2	T_3	T_4	Tiempo disponible
O_1	2	3	4	1	4
O_2	3	2	3	2	5
O_3	2	2	1	2	3
O_4	3	3	3	3	4
O_5	2	1	2	2	4
Tiempo requerido	3	2	2	3	

Implementación (I) – Mono-Objetivo

- Se optimiza **un único objetivo**, reducir el `total_cost`
- Todas las hormigas usan **una sola función de coste** y una única feromona.
- Solo la mejor hormiga `best_global` deposita feromonas al final de cada generación para reforzar el camino de coste mínimo
- Sirve como **baseline** para comparar con MOACO y MAS.

Implementación (I) – Mono-Objetivo

Salida del algoritmo

=== 1. Ejecutando Mono-Objetivo ===

-> Tiempo: 0.0248 seg

Mejor Solución: Coste 6, Asignación [0, 4, 2, 1]

**Explicación de la
solución:**

Asignación de tareas	Coste
Tarea 1 a Operario 1	2
Tarea 2 a Operario 5	1
Tarea 3 a Opearario 3	1
Tarea 4 a Opearario 2	2
Total	6

Implementación (II) – MOACO

- Se busca minimizar simultáneamente el Coste Total y el Desequilibrio de Carga
- El resultado no es una sola ruta, sino un **frente de Pareto** de soluciones “igual de buenas” según distintos criterios.
- Para esto hemos creado la función `update_pareto` que filtra soluciones
- Permite **ajustar las preferencias** según el escenario (pesos, filtros, etc.).

Implementación (II) – MOACO

Salida del algoritmo

-> Tiempo: 1.5261 seg

Soluciones en Frente: 2

--- Tabla de Soluciones (Simple MOACO) ---

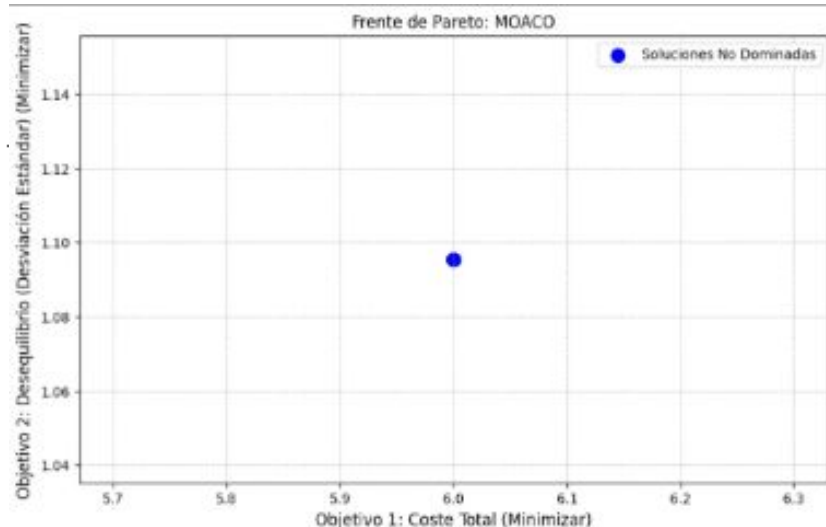
Coste	StdDev	Asignación
-------	--------	------------

6	1.0954	[4, 1, 2, 0]
---	--------	--------------

6	1.0954	[0, 4, 2, 1]
---	--------	--------------



2 posibles soluciones



Implementación (III) – MAS

- Cada hormiga no es igual; se le asigna un "perfil de preferencia" λ único al nacer. Unas son "ahorradoras" (priorizan coste) y otras "equilibradoras" (priorizan balance de carga).

```
self.lambda_weight = ant_id / (num_ants - 1)
```

- La gran diferencia es que la probabilidad de elegir una tarea combina dos heurísticas h_1 y h_2 ponderadas por ese λ .
 - h_1 (Estática): Inverso del Coste (Prefiere lo barato)
 - h_2 (Dinámica): Tiempo Libre Restante del operario (Prefiere equilibrar la carga para que nadie se sature).
- Gracias a estos pesos distintos, la colonia no converge a un solo punto, sino que se "despliega" para cubrir todo el abanico de soluciones posibles (desde las muy baratas pero desequilibradas, hasta las muy equilibradas pero caras).

Implementación (III) – MAS

Salida del algoritmo

-> Tiempo: 3.3758 seg

Soluciones en Frente: 4

--- Tabla de Soluciones (MAS) ---

Coste	StdDev	Asignación
-------	--------	------------

6	1.0954	[1, 4, 2, 0]
---	--------	--------------

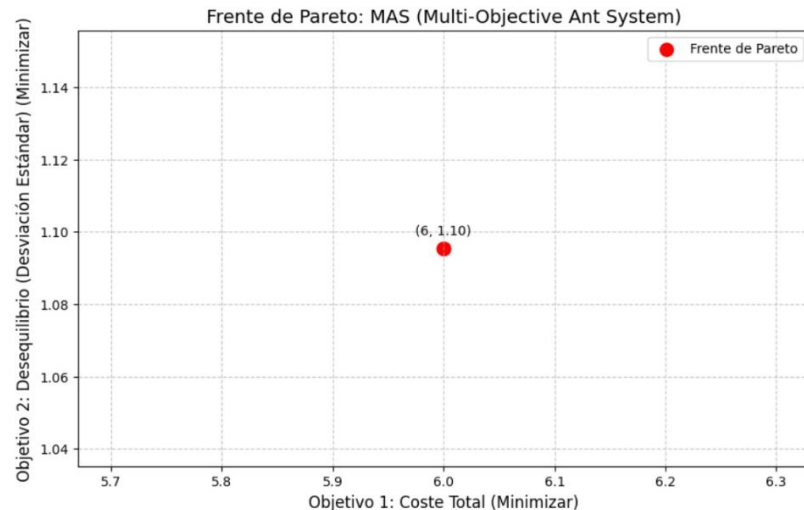
6	1.0954	[4, 1, 2, 0]
---	--------	--------------

6	1.0954	[0, 4, 2, 1]
---	--------	--------------

6	1.0954	[3, 4, 2, 0]
---	--------	--------------

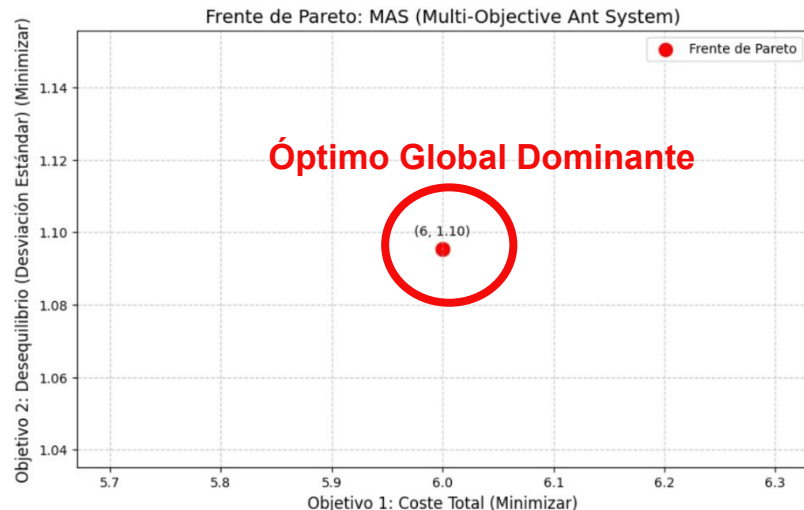


4 posibles soluciones



Convergencia del Frente de Pareto: Óptimo Global Único

A pesar de buscar un compromiso (trade-off) entre Coste y Equilibrio, los datos del enunciado (4 tareas, 5 operarios) hacen que el Frente de Pareto converja a un único punto óptimo

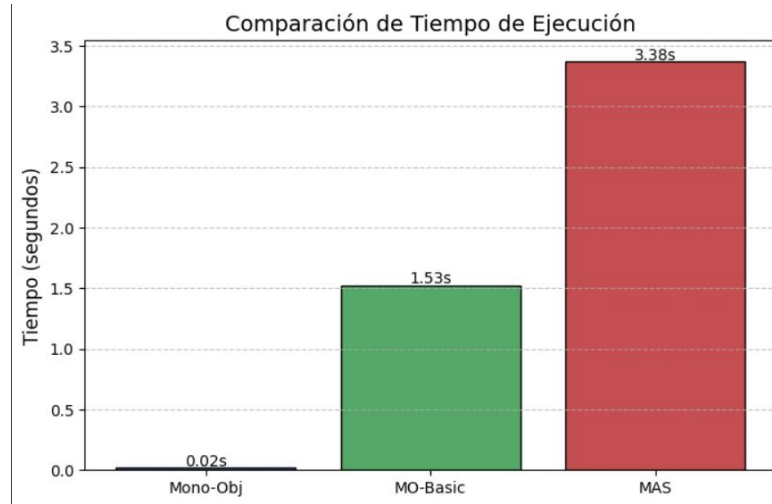


Punto Óptimo: (**Coste: 6, StdDev: 1.0954**).

El algoritmo encuentra **múltiples asignaciones diferentes** (distintos operarios haciendo distintas tareas).

6	1.0954	[1, 4, 2, 0]
6	1.0954	[4, 1, 2, 0]
6	1.0954	[0, 4, 2, 1]
6	1.0954	[3, 4, 2, 0]

Implementación (IV) – Diferencias



MAS es computacionalmente más costoso porque debe calcular probabilidades dinámicas h_2 para cada hormiga individualmente, mientras que Mono-Objetivo usa una heurística estática pre-calculada

Conclusiones

